Algorithm 1 Proposed Drift Detection Method

Result: Estimated Drift, Samples Causing Drift
Inputs:
D_1 : Reference distribution (e.g. set of embeddings)
D_2: Target distribution (e.g. set of embeddings)
Variable:
β : Number of samples to be compared
M : Total number of samples
K : Number of bootstraps
$D_{1,j}$: jth sample of reference distribution
$D_{2,j}$: jth sample of target distribution
R : List of MMD statistics
R_b : List of bootstraped MMD statistics
X : List of Median MMD statistics
t : Index
for $t = \beta : M$ **do**
  $Q_1 = \{D_{1,1}, D_{1,2}, ..., D_{1,t}\}$ ▷ batch of reference distribution
  $Q_2 = \{D_{2,1}, D_{2,2}, ..., D_{2,t}\}$ ▷ batch of target distribution
  $r = MMD(Q_1, Q_2)$ ▷ compute Maximum Mean Discrepancy
  $T = \{Q_1, Q_2\}$ ▷ combine Q_1 and Q_2 (under H_0 of no drift)
  $R = \{R; r\}$ ▷ append MMD statistic

  for $i = 1 : K$ **do**
    $T^{'} = Bootstrap(T)$ ▷ get a bootstrap samples from T
    $Q_1^{'} = T^{'}_{1:\beta/2}$ ▷ get 1st half of $T^{'}$
    $Q_2^{'} = T^{'}_{\beta/2+1:\beta}$ ▷ get 2nd half of $T^{'}$
    $r^{'} = MMD(Q_1^{'}, Q_2^{'})$
    $R_b = \{R_b; r^{'}\}$ ▷ append MMD statistic
  end for
  $d_{med} = \textit{Median}(R_b)$ ▷ compute median of estimated MMDs
  $X = \{X; d_{med}\}$ ▷ append d_{med} to X
  $t+ = 1$ ▷ increase index
  $Q_1 = \{Q_1; D_{1,t}\}$ ▷ add next sample to Q_1
  $Q_2 = \{Q_2; D_{2,t}\}$ ▷ add next sample to Q_2
  Q_1.*popleft()* ▷ remove first sample from Q_1
  Q_2.*popleft()* ▷ remove first sample from Q_2
end for
z=$Argmax(R)$ ▷ highest drift Index
$U = \{D_{l,z-\beta}, D_{l,z-\beta+1}, ..., D_{l,z}\} \quad l \in \{1, 2\}$
Return $d = \textit{mean}(X)$ ▷ median MMD
Return U ▷ samples causing highest drift

The results in Table 1 demonstrate a significant correlation between estimated drift (MMD) and model performance metrics. Consequently, our unsupervised drift detection approach can effectively monitor model performance and accurately predict model regression. This capability is particularly valuable in production environments where annotated data may not be available, making our drift detection method the only practical option for model performance monitoring. Figure 2 depicts the estimated drift and model performance over time, revealing a notable increase in drift as time progresses. This increase is attributed to the emergence of new data patterns in the production data, causing an increase in the estimated drift and a corresponding drop in the model's performance. These findings underscore the need for ongoing monitoring and recalibration of the model to ensure its continued effectiveness.

3.2 Model Performance Regression Mitigation

To assess the effectiveness of our proposed drift detection method in reducing the decline in model performance from drift when used in production, we outline a comprehensive mitigation process. Firstly, we run our drift detection method by setting the reference and target distributions to be the model's training and inference data during production,

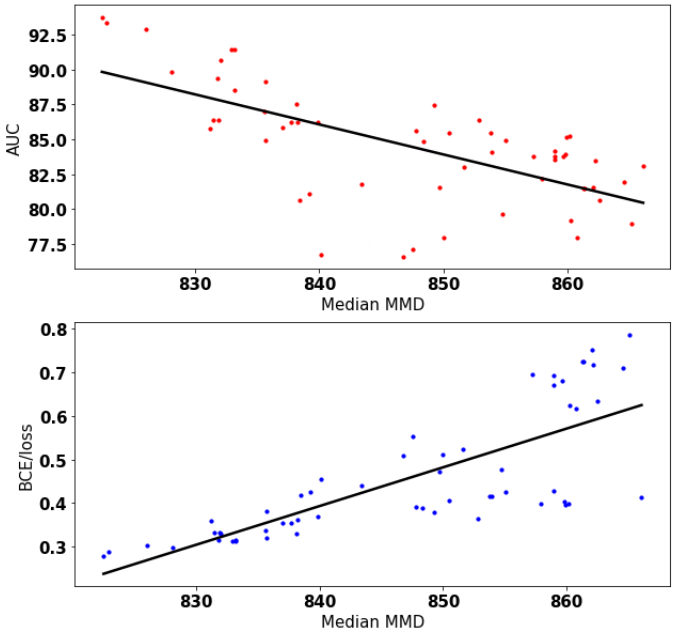

Figure 1: Plot of estimated drift (MMD) vs the model performance metrics for monthly buckets, which indicates that as the amount of drift increases, model performance drops.

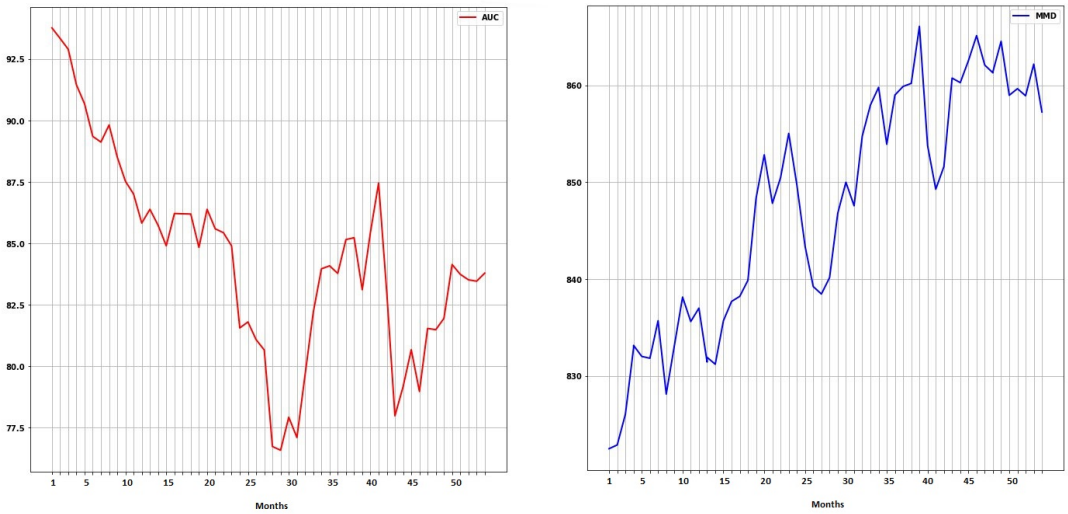

Figure 2: Left and right plots shows the model performance metric (AUC) and the estimated amount of drift (MMD), respectively. The increase in MMD over time is attributed to the emergence of new data patterns in the production data, causing an increase in the estimated drift and a corresponding drop in the model's performance

respectively. Then, our proposed method identifies the samples that exhibit the highest degree of drift or deviation from the production data. Finally, we incorporate the dataset generated from the previous step into the model's training data and retrain the model, enabling it to improve its performance during production.

We implemented the above-mentioned approach on a multi-task model used for domain classification, intent classification, and name entity recognition. The model architecture comprises a blend of Bi-LSTM and transformer models [Devlin et al., 2018].

To improve this model performance, we employed our proposed drift detection method to compare the model's training data with one month of production data. This enabled us to identify samples with the highest degree of drift from production data. However, in order to use these data during model training, they need to be annotated. Therefore, we

utilized a semi-supervised approach, where we used a large transformer model that leverages a variety of offline signals such as previous conversations that are not available during online inference, to generate pseudo-labels for these data. With the help of these pseudo-labels, we added new samples with highest amount of drift to the model's training data and retrained the model.

We conducted a thorough evaluation of the new model trained on samples generated from our drift detection method. Specifically, we tested the model's performance on a held-out false accept (FA) dataset, consisting of only false accept data, to evaluate its ability to reject false accept samples. Additionally, we compared the performance of our proposed approach with several common mitigation methods. These methods include the following:

Baseline: A baseline model trained using the model's original training data without any mitigation.

Bias reduction: This method involves clustering the training and production data to identify patterns that are absent within the training data. We then added new samples from absent patterns to the training data using this approach.

Upsampling: This method first identifies the underrepresented samples within the training data by clustering and then employs upsampling to increase their frequency in the training data. Then, we added underrepresented samples to the training data using this approach.

We re-trained the model using all these methods, and Table 2 provides a detailed comparison of the various methods used in this experiment. The results indicate that the samples of data added by our proposed drift detection method significantly improve model performance and outperform other methods to varying degrees, without an increase in false reject rate.

Method	FAR (%)
Baseline	73.15
Bias reduction	61.85
Upsampling	71.48
Proposed drift detection	**59.68**

Table 2: Comparison of different mitigation methods on false accept rate (FAR) test set. Our proposed drift detection method shows superior performance compared to the other methods. Lower FAR values indicate better performance.

3.3 Encoder Effect Ablation Study

In this section, we outline an experiment designed to assess the effect of different encoders on the performance of our proposed method. We utilize different encoders to extract text embeddings and focus on simulating data drift within a binary classification scenario. The reference dataset initially maintains a balanced distribution of positive and negative class instances, each accounting for 50% of the dataset. Subsequently, we change the positive class percentage within the target dataset to induce data drift. Following datasets are used in our experiment:

AG news: AG news comprises over one million news articles, compiled from a diverse range of 2000+ news sources over a span of more than a year [Zhang et al., 2015]. We use world and sports classes in our experiment.

Yelp review: yelp review dataset consists of reviews extracted from the Yelp [Zhang et al., 2016]. We use reviews rated 5 as the positive class and reviews rated 1 as the negative class in our experiment.

We sample 5000 observations from each dataset where we increase/decrease the percentage of positive class in the target dataset while comparing it to the reference dataset with equal ratio of positive and negative classes. We set the batch size, β, and bootstrap steps of our method to be 64, 32, and 50, respectively. We use the following encoders in our experiment:

bert-base-uncased: BERT base [Devlin et al., 2018] is a transformer encoder model with 12 layers with 768 hidden size that is pretrained on a large corpus of English data.

bert-large-uncased: this is a BERT model with 24 layers with 1024 hidden size.

all-MiniLM-L12-v2: This model, known as a sentence-transformers model, effectively encodes sentences and paragraphs into a 384-dimensional dense vector space, offering utility in applications such as clustering or semantic search [SBERT, 2023]. This model has 12 layers with 384 hidden size.

all-distilroberta-v1: This model is also a sentence-transformer with 12 layers and 768 hidden size.

We ran our drift detection method using all four different encoders on both datasets and the following Figure 3 shows the results. The results indicate that there is a small amount of drift when the percentage of positive class is close to